

RV32I BASE INTEGER INSTRUCTIONS, in alphabetical order

MNEMONIC	FMT	NAME	DESCRIPTION (in Verilog)	NOTE
add	R	ADD	$R[rd] = R[rs1] + R[rs2]$	
addi	I	ADD Immediate	$R[rd] = R[rs1] + imm$	
and	R	AND	$R[rd] = R[rs1] \& R[rs2]$	
andi	I	AND Immediate	$R[rd] = R[rs1] \& imm$	
auipc	U	Add Upper Immediate to PC	$R[rd] = PC + \{imm, 12'b0\}$	
beq	SB	Branch Equal	if($R[rs1] == R[rs2]$) $PC = PC + \{imm, 1b'0\}$	
bge	SB	Branch Greater than or Equal	if($R[rs1] \geq R[rs2]$) $PC = PC + \{imm, 1b'0\}$	
bgeu	SB	Branch $\geq$ Unsigned	if($R[rs1] \geq_u R[rs2]$) $PC = PC + \{imm, 1b'0\}$	2)
blt	SB	Branch Less Than	if($R[rs1] < R[rs2]$) $PC = PC + \{imm, 1b'0\}$	
bltu	SB	Branch Less Than Unsigned	if($R[rs1] <_u R[rs2]$) $PC = PC + \{imm, 1b'0\}$	2)
bne	SB	Branch Not Equal	if($R[rs1] != R[rs2]$) $PC = PC + \{imm, 1b'0\}$	
csrrc	I	Cont./Stat.RegRead&Clear	$R[rd] = CSR; CSR = CSR \& \sim R[rs1]$	
csrrci	I	Cont./Stat.RegRead&Clear Imm	$R[rd] = CSR; CSR = CSR \& \sim imm$	
csrrs	I	Cont./Stat.RegRead&Set	$R[rd] = CSR; CSR = CSR R[rs1]$	
csrrsi	I	Cont./Stat.RegRead&Set Imm	$R[rd] = CSR; CSR = CSR imm$	
csrrw	I	Cont./Stat.RegRead&Write	$R[rd] = CSR; CSR = R[rs1]$	
csrrwi	I	Cont./Stat.Reg Read&Write Imm	$R[rd] = CSR; CSR = imm$	
ebreak	I	Environment BREAK	Transfer control to debugger	
ecall	I	Environment CALL	Transfer control to operating system	
fence	I	Synch thread	Synchronizes threads	
fence.i	I	Synch Instr & Data	Synchronizes writes to instruction stream	
jal	UJ	Jump & Link	$R[rd] = PC + 4; PC = PC + \{imm, 1b'0\}$	
jalr	I	Jump & Link Register	$R[rd] = PC + 4; PC = R[rs1] + imm$	
lb	I	Load Byte	$R[rd] = \{24'bM\}(7), M[R[rs1] + imm](7:0)\}$	3) 4)
lbu	I	Load Byte Unsigned	$R[rd] = \{24'b0, M[R[rs1] + imm](7:0)\}$	
lh	I	Load Halfword	$R[rd] = \{16'bM\}(15), M[R[rs1] + imm](15:0)\}$	
lhu	I	Load Halfword Unsigned	$R[rd] = \{16'b0, M[R[rs1] + imm](15:0)\}$	4)
lui	U	Load Upper Immediate	$R[rd] = \{imm, 12'b0\}$	
lw	I	Load Word	$R[rd] = \{M[R[rs1] + imm](31:0)\}$	
or	R	OR	$R[rd] = R[rs1] R[rs2]$	
ori	I	OR Immediate	$R[rd] = R[rs1] imm$	4)
sb	S	Store Byte	$M[R[rs1] + imm](7:0) = R[rs2](7:0)$	
sh	S	Store Halfword	$M[R[rs1] + imm](15:0) = R[rs2](15:0)$	
sll	R	Shift Left	$R[rd] = R[rs1] \ll R[rs2]$	
slli	I	Shift Left Immediate	$R[rd] = R[rs1] \ll imm$	
slt	R	Set Less Than	$R[rd] = (R[rs1] < R[rs2]) ? 1 : 0$	
slti	I	Set Less Than Immediate	$R[rd] = (R[rs1] < imm) ? 1 : 0$	
sltiu	I	Set < Immediate Unsigned	$R[rd] = (R[rs1] <_u imm) ? 1 : 0$	
sltu	R	Set Less Than Unsigned	$R[rd] = (R[rs1] <_u R[rs2]) ? 1 : 0$	
sra	R	Shift Right Arithmetic	$R[rd] = R[rs1] \gg R[rs2]$	
srai	I	Shift Right Arith Imm	$R[rd] = R[rs1] \gg imm$	2)
srl	R	Shift Right (Word)	$R[rd] = R[rs1] \gg R[rs2]$	2)
srli	I	Shift Right Immediate	$R[rd] = R[rs1] \gg imm$	5)
sub, subw	R	SUBtract (Word)	$R[rd] = R[rs1] - R[rs2]$	5)
sw	S	Store Word	$M[R[rs1] + imm](31:0) = R[rs2](31:0)$	
xor	R	XOR	$R[rd] = R[rs1] \wedge R[rs2]$	
xori	I	XOR Immediate	$R[rd] = R[rs1] \wedge imm$	

- Notes: 1) Operation assumes unsigned integers (instead of 2's complement)
 - 2) The least significant bit of the branch address in jalr is set to 0
 - 3) (signed) Load instructions extend the sign bit of data to fill the 32-bit register
 - 4) Replicates the sign bit to fill in the leftmost bits of the result during right shift
 - 5) Multiply with one operand signed and one unsigned
 - 6) The Single version does a single-precision operation using the rightmost 32 bits of a 64-bit F register
 - 7) Classify writes a 10-bit mask to show which properties are true (e.g., -inf, -0, +0, +inf, denorm, ...)
 - 8) Atomic memory operation; nothing else can interpose itself between the read and the write of the memory location
- The immediate field is sign-extended in RISC-V

1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027, 2028, 2029, 2030, 2031, 2032, 2033, 2034, 2035, 2036, 2037, 2038, 2039, 2040, 2041, 2042, 2043, 2044, 2045, 2046, 2047, 2048, 2049, 2050, 2051, 2052, 2053, 2054, 2055, 2056, 2057, 2058, 2059, 2060, 2061, 2062, 2063, 2064, 2065, 2066, 2067, 2068, 2069, 2070, 2071, 2072, 2073, 2074, 2075, 2076, 2077, 2078, 2079, 2080, 2081, 2082, 2083, 2084, 2085, 2086, 2087, 2088, 2089, 2090, 2091, 2092, 2093, 2094, 2095, 2096, 2097, 2098, 2099, 2100, 2101, 2102, 2103, 2104, 2105, 2106, 2107, 2108, 2109, 2110, 2111, 2112, 2113, 2114, 2115, 2116, 2117, 2118, 2119, 2120, 2121, 2122, 2123, 2124, 2125, 2126, 2127, 2128, 2129, 2130, 2131, 2132, 2133, 2134, 2135, 2136, 2137, 2138, 2139, 2140, 2141, 2142, 2143, 2144, 2145, 2146, 2147, 2148, 2149, 2150, 2151, 2152, 2153, 2154, 2155, 2156, 2157, 2158, 2159, 2160, 2161, 2162, 2163, 2164, 2165, 2166, 2167, 2168, 2169, 2170, 2171, 2172, 2173, 2174, 2175, 2176, 2177, 2178, 2179, 2180, 2181, 2182, 2183, 2184, 2185, 2186, 2187, 2188, 2189, 2190, 2191, 2192, 2193, 2194, 2195, 2196, 2197, 2198, 2199, 2200, 2201, 2202, 2203, 2204, 2205, 2206, 2207, 2208, 2209, 2210, 2211, 2212, 2213, 2214, 2215, 2216, 2217, 2218, 2219, 2220, 2221, 2222, 2223, 2224, 2225, 2226, 2227, 2228, 2229, 2230, 2231, 2232, 2233, 2234, 2235, 2236, 2237, 2238, 2239, 2240, 2241, 2242, 2243, 2244, 2245, 2246, 2247, 2248, 2249, 2250, 2251, 2252, 2253, 2254, 2255, 2256, 2257, 2258, 2259, 2260, 2261, 2262, 2263, 2264, 2265, 2266, 2267, 2268, 2269, 2270, 2271, 2272, 2273, 2274, 2275, 2276, 2277, 2278, 2279, 2280, 2281, 2282, 2283, 2284, 2285, 2286, 2287, 2288, 2289, 2290, 2291, 2292, 2293, 2294, 2295, 2296, 2297, 2298, 2299, 2300, 2301, 2302, 2303, 2304, 2305, 2306, 2307, 2308, 2309, 2310, 2311, 2312, 2313, 2314, 2315, 2316, 2317, 2318, 2319, 2320, 2321, 2322, 2323, 2324, 2325, 2326, 2327, 2328, 2329, 2330, 2331, 2332, 2333, 2334, 2335, 2336, 2337, 2338, 2339, 2340, 2341, 2342, 2343, 2344, 2345, 2346, 2347, 2348, 2349, 2350, 2351, 2352, 2353, 2354, 2355, 2356, 2357, 2358, 2359, 2360, 2361, 2362, 2363, 2364, 2365, 2366, 2367, 2368, 2369, 2370, 2371, 2372, 2373, 2374, 2375, 2376, 2377, 2378, 2379, 2380, 2381, 2382, 2383, 2384, 2385, 2386, 2387, 2388, 2389, 2390, 2391, 2392, 2393, 2394, 2395, 2396, 2397, 2398, 2399, 2400, 2401, 2402, 2403, 2404, 2405, 2406, 2407, 2408, 2409, 2410, 2411, 2412, 2413, 2414, 2415, 2416, 2417, 2418, 2419, 2420, 2421, 2422, 2423, 2424, 2425, 2426, 2427, 2428, 2429, 2430, 2431, 2432, 2433, 2434, 2435, 2436, 2437, 2438, 2439, 2440, 2441, 2442, 2443, 2444, 2445, 2446, 2447, 2448, 2449, 2450, 2451, 2452, 2453, 2454, 2455, 2456, 2457, 2458, 2459, 2460, 2461, 2462, 2463, 2464, 2465, 2466, 2467, 2468, 2469, 2470, 2471, 2472, 2473, 2474, 2475, 2476, 2477, 2478, 2479, 2480, 2481, 2482, 2483, 2484, 2485, 2486, 2487, 2488, 2489, 2490, 2491, 2492, 2493, 2494, 2495, 2496, 2497, 2498, 2499, 2500, 2501, 2502, 2503, 2504, 2505, 2506, 2507, 2508, 2509, 2510, 2511, 2512, 2513, 2514, 2515, 2516, 2517, 2518, 2519, 2520, 2521, 2522, 2523, 2524, 2525, 2526, 2527, 2528, 2529, 2530, 2531, 2532, 2533, 2534, 2535, 2536, 2537, 2538, 2539, 2540, 2541, 2542, 2543, 2544, 2545, 2546, 2547, 2548, 2549, 2550, 2551, 2552, 2553, 2554, 2555, 2556, 2557, 2558, 2559, 2560, 2561, 2562, 2563, 2564, 2565, 2566, 2567, 2568, 2569, 2570, 2571, 2572, 2573, 2574, 2575, 2576, 2577, 2578, 2579, 2580, 2581, 2582, 2583, 2584, 2585, 2586, 2587, 2588, 2589, 2590, 2591, 2592, 2593, 2594, 2595, 2596, 2597, 2598, 2599, 2600, 2601, 2602, 2603, 2604, 2605, 2606, 2607, 2608, 2609, 2610, 2611, 2612, 2613, 2614, 2615, 2616, 2617, 2618, 2619, 2620, 2621, 2622, 2623, 2624, 2625, 2626, 2627, 2628, 2629, 2630, 2631, 2632, 2633, 2634, 2635, 2636, 2637, 2638, 2639, 2640, 2641, 2642, 2643, 2644, 2645, 2646, 2647, 2648, 2649, 2650, 2651, 2652, 2653, 2654, 2655, 2656, 2657, 2658, 2659, 2660, 2661, 2662, 2663, 2664, 2665, 2666, 2667, 2668, 2669, 2670, 2671, 2672, 2673, 2674, 2675, 2676, 2677, 2678, 2679, 2680, 26

MNEMONIC	FMT NAME	DESCRIPTION (in Verilog)	NOTE
mul	R MULtiple	$R[rd] = (R[rs1] * R[rs2])(63:0)$	
mulh	R MULtiple High	$R[rd] = (R[rs1] * R[rs2])(127:64)$	
mulhsu	R MULtiple High Unsigned	$R[rd] = (R[rs1] * R[rs2])(127:64)$	2)
mulhu	R MULtiple upper Half Unsigned	$R[rd] = (R[rs1] * R[rs2])(127:64)$	6)
div	R DIvide	$R[rd] = (R[rs1] / R[rs2])$	
divu	R DIvide Unsigned	$R[rd] = (R[rs1] / R[rs2])$	2)
rem	R REMainder	$R[rd] = (R[rs1] \% R[rs2])$	
remu	R REMainder Unsigned	$R[rd] = (R[rs1] \% R[rs2])$	2)

RV64F and RV64D Floating-Point Extensions

Instruction	Operation	Formula	Category
fld, flw	I Load (Word)	$F[rd] = M[R[rs1] + imm]$	
fsd, fsw	S Store (Word)	$M[R[rs1] + imm] = F[rd]$	
fadd.s, fadd.d	R ADD	$F[rd] = F[rs1] + F[rs2]$	7)
fsub.s, fsub.d	R SUBtract	$F[rd] = F[rs1] - F[rs2]$	7)
fmul.s, fmul.d	R MULtiply	$F[rd] = F[rs1] * F[rs2]$	7)
fdiv.s, fdiv.d	R DIvide	$F[rd] = F[rs1] / F[rs2]$	7)
fsqrt.s, fsqrt.d	R SQare RooT	$F[rd] = \sqrt{F[rs1]}$	7)
fmadd.s, fmadd.d	R Multiply-ADD	$F[rd] = F[rs1] * F[rs2] + F[rs3]$	7)
fmsub.s, fmsub.d	R Multiply-SUBtract	$F[rd] = F[rs1] * F[rs2] - F[rs3]$	7)
fmsub.s, fmsub.d	R Negative Multiply-ADD	$F[rd] = -(F[rs1] * F[rs2] - F[rs3])$	7)
fmnadd.s, fmnadd.d	R Negative Multiply-SUBtract	$F[rd] = -(F[rs1] * F[rs2] + F[rs3])$	7)
fsgnj.s, fsgnj.d	R SiGN source	$F[rd] = \{ F[rs2] < 63, F[rs1] < 62:0 \}$	7)
fsgnjn.s, fsgnjn.d	R Negative SiGN source	$F[rd] = \{ (-F[rs2] < 63), F[rs1] < 62:0 \}$	7)
fsgnjx.s, fsgnjx.d	R Xor SiGN source	$F[rd] = \{ F[rs2] < 63 \oplus F[rs1] < 63, F[rs1] < 62:0 \}$	7)
fmin.s, fmin.d	R MiNimum	$F[rd] = (F[rs1] < F[rs2]) ? F[rs1] : F[rs2]$	7)
fmax.s, fmax.d	R MAximum	$F[rd] = (F[rs1] > F[rs2]) ? F[rs1] : F[rs2]$	7)
feq.s, feq.d	R Compare Float EQual	$R[rd] = (F[rs1] == F[rs2]) ? 1 : 0$	7)
flt.s, flt.d	R Compare Float Less Than	$R[rd] = (F[rs1] < F[rs2]) ? 1 : 0$	7)
fle.s, fle.d	R Compare Float Less than or =	$R[rd] = (F[rs1] <= F[rs2]) ? 1 : 0$	7)
fclass.s, fclass.d	R Classify Type	$R[rd] = \text{class}(F[rs1])$	7, 8)
fmv.x.s, fmv.d.x	R Move from 64b Integer	$F[rd] = R[rs1]$	7)
fmv.x.s, fmv.x.d	R Move to Integer	$R[rd] = F[rs1]$	7)
fcvt.d.s	R Convert from SP to DP	$F[rd] = \text{single}(F[rs1])$	
fcvt.s.d	R Convert from DP to SP	$F[rd] = \text{double}(F[rs1])$	
fcvt.s.w, fcvt.d.w	R Convert from 32b Integer	$F[rd] = \text{float}(R[rs1])(31:0)$	7)
fcvt.s.l, fcvt.d.l	R Convert from 64b Integer	$F[rd] = \text{float}(R[rs1])(63:0)$	7)
fcvt.s.wu, fcvt.d.wu	R Convert from 32b Int Unsigned	$F[rd] = \text{float}(R[rs1])(31:0)$	2, 7)
fcvt.s.lu, fcvt.d.lu	R Convert from 64b Int Unsigned	$F[rd] = \text{float}(R[rs1])(63:0)$	2, 7)
fcvt.w.s, fcvt.w.d	R Convert to 32b Integer	$R[rd](31:0) = \text{integer}(F[rs1])$	7)
fcvt.l.s, fcvt.l.d	R Convert to 64b Integer	$R[rd](63:0) = \text{integer}(F[rs1])$	7)
fcvt.wu.s, fcvt.wu.d	R Convert to 32b Int Unsigned	$R[rd](31:0) = \text{integer}(F[rs1])$	2, 7)
fcvt.lu.s, fcvt.lu.d	R Convert to 64b Int Unsigned	$R[rd](63:0) = \text{integer}(F[rs1])$	2, 7)

RV64A Atomic Extension

amoadd.w, amoadd.d	R	ADD	$R[rd] = M[R[rs1]],$ $M[R[rs1]] = M[R[rs1]] + R[rs2]$	9)
amoand.w, amoand.d	R	AND	$R[rd] = M[R[rs1]],$ $M[R[rs1]] = M[R[rs1]] \& R[rs2]$	9)
amomax.w, amomax.d	R	MAXimum	$R[rd] = M[R[rs1]],$ $\text{if } (R[rs2] > M[R[rs1]]) \text{ } M[R[rs1]] = R[rs2]$	9)
amomaxu.w, amomaxu.d	R	MAXimum Unsigned	$R[rd] = M[R[rs1]],$ $\text{if } (R[rs2] > M[R[rs1]]) \text{ } M[R[rs1]] = R[rs2]$	2,9)
amomin.w, amomin.d	R	MINimum	$R[rd] = M[R[rs1]],$ $\text{if } (R[rs2] < M[R[rs1]]) \text{ } M[R[rs1]] = R[rs2]$	9)
amominu.w, amominu.d	R	MINimum Unsigned	$R[rd] = M[R[rs1]],$ $\text{if } (R[rs2] < M[R[rs1]]) \text{ } M[R[rs1]] = R[rs2]$	2,9)
amoor.w, amoor.d	R	OR	$R[rd] = M[R[rs1]],$ $M[R[rs1]] = M[R[rs1]] R[rs2]$	9)
amoswap.w, amoswap.d	R	SWAP	$R[rd] = M[R[rs1]], \text{ } M[R[rs1]] = R[rs2]$	9)
amoxor.w, amoxor.d	R	XOR	$R[rd] = M[R[rs1]],$ $M[R[rs1]] = M[R[rs1]] \wedge R[rs2]$	9)
lr.w, lr.d	R	Load Reserved	$R[rd] = M[R[rs1]],$ $\text{reservation on } M[R[rs1]]$	
sc.w, sc.d	R	Store Conditional	$\text{if reserved, } M[R[rs1]] = R[rs2],$ $R[rd] = 0; \text{ else } R[rd] = 1$	

CORE INSTRUCTION FORMATS

	31	27	26	25	24	20	19	15	14	12	11	7	6	0
R	funct7				rs2		rs1		funct3		rd		Opcode	
I	imm[11:0]						rs1		funct3		rd		Opcode	
S	imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode	
SB	imm[12:10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode	
U	imm[31:12]										rd		opcode	
UJ	imm[20 10:1 11 19:12]										rd		opcode	

PCODES IN NUMERICAL ORDER BY OPCODE

MNEMONIC	NAME	DESCRIPTION	USES
beqz	Branch = zero	if(R[rs1]==0) PC=PC+{imm,1b'0}	beq
bnez	Branch ≠ zero	if(R[rs1]! =0) PC=PC+{imm,1b'0}	bne
fabs.s, fabs.d	Absolute Value	F[rd] = (F[rs1]< 0) ? -F[rs1] : F[rs1]	fsgnx
fmv.s, fmv.d	FP Move	F[rd] = F[rs1]	fsgnj
fneg.s, fneg.d	FP negate	F[rd] = -F[rs1]	fsgnjn
j	Jump	PC = {imm,1b'0}	jal
jr	Jump register	PC = R[rs1]	jalr
la	Load address	R[rd] = address	auipc
li	Load imm	R[rd] = imm	addi
mv	Move	R[rd] = R[rs1]	addi
neg	Negate	R[rd] = -R[rs1]	sub
nop	No operation	R[0] = R[0]	addi
not	Not	R[rd] = ~R[rs1]	xori
ret	Return	PC = R[1]	jalr
seqz	Set = zero	R[rd] = (R[rs1]== 0) ? 1 : 0	sltiu
snez	Set ≠ zero	R[rd] = (R[rs1]! =0) ? 1 : 0	sltu

OPCODES IN NUMERICAL ORDER BY OPCODE

MNEMONIC	FMT	OPCODE	FUNCT3	FUNCT7 OR IMM	HEXADECEMAL
lb	I	0000011	000		03/0
lh	I	0000011	001		03/1
lw	I	0000011	010		03/2

lbz	I	0000011	100		03/4
lhz	I	0000011	101		03/5

fence	I	0001111	000		0F/0
fence.i	I	0001111	001		0F/1
addi	I	0010011	000		13/0
slli	I	0010011	001	0000000	13/1/00
slti	I	0010011	010		13/2
sltiu	I	0010011	011		13/3
xori	I	0010011	100		13/4
srlr	I	0010011	101	0000000	13/5/00
srai	I	0010011	101	0100000	13/5/20
ori	I	0010011	110		13/6
andi	I	0010011	111		13/7
auipc	U	0010111			17

sb	S	0100011	000		23/0
sh	S	0100011	001		23/1
sw	S	0100011	010		23/2

add	R	0110011	000	0000000	33/0/00
sub	R	0110011	000	0100000	33/0/20
sll	R	0110011	001	0000000	33/1/00
slt	R	0110011	010	0000000	33/2/00
sltu	R	0110011	011	0000000	33/3/00
xor	R	0110011	100	0000000	33/4/00
srl	R	0110011	101	0000000	33/5/00
sra	R	0110011	101	0100000	33/5/20
or	R	0110011	110	0000000	33/6/00
and	R	0110011	111	0000000	33/7/00
lui	U	0110111			37

beq	SB	1100011	000		63/0
bne	SB	1100011	001		63/1
blt	SB	1100011	100		63/4
bge	SB	1100011	101		63/5
bltu	SB	1100011	110		63/6
bgeu	SB	1100011	111		63/7
jalr	I	1100111	000		67/0
jal	UJ	1101111			6F
ecall	I	1110011	000	000000000000	73/0/000
ebreak	I	1110011	000	000000000001	73/0/001
CSRRW	I	1110011	001		73/1
CSRRS	I	1110011	010		73/2
CSRRC	I	1110011	011		73/3
CSRRWI	I	1110011	101		73/5
CSRRSI	I	1110011	110		73/6
CSRRCI	I	1110011	111		73/7

REGISTER NAME, USE, CALLING CONVENTION

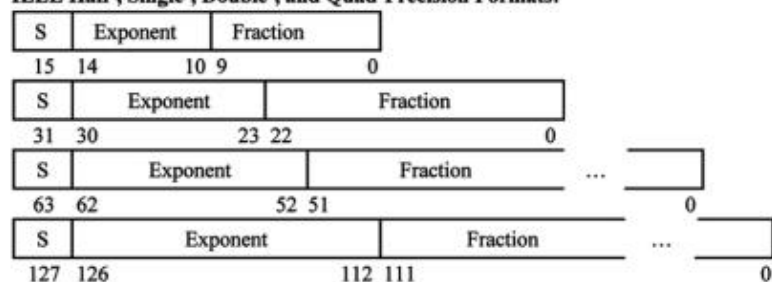
REGISTER	NAME	USE	SAVER
x0	zero	The constant value 0	N.A.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	--
x4	tp	Thread pointer	--
x5~x7	t0~t2	Temporaries	Caller
x8	s0/ fp	Saved register/Frame pointer	Callee
x9	s1	Saved register	Callee
x10~x11	a0~a1	Function arguments/Return values	Caller
x12~x17	a2~a7	Function arguments	Caller
x18~x27	s2~s11	Saved registers	Callee
x28~x31	t3~t6	Temporaries	Caller
f0~f7	ft0~ft7	FP Temporaries	Caller
f8~f9	fs0~fs1	FP Saved registers	Callee
f10~f11	fa0~fa1	FP Function arguments/Return values	Caller
f12~f17	fa2~fa7	FP Function arguments	Caller
f18~f27	fs2~fs11	FP Saved registers	Callee
f28~f31	ft8~ft11	R[rd] = R[rs1] + R[rs2]	Caller

IEEE 754 FLOATING-POINT STANDARD

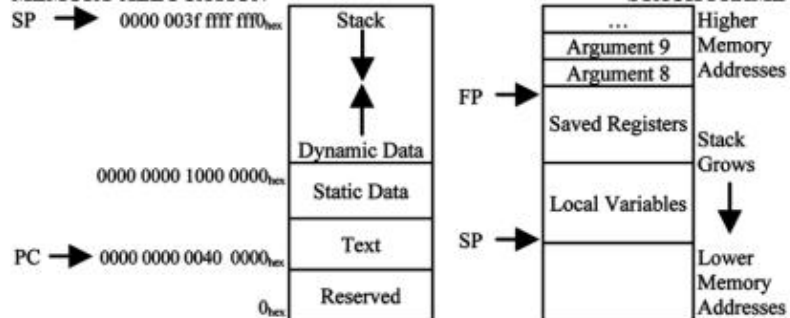
$$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

where Half-Precision Bias = 15, Single-Precision Bias = 127, Double-Precision Bias = 1023, Quad-Precision Bias = 16383

IEEE Half-, Single-, Double-, and Quad-Precision Formats:



MEMORY ALLOCATION



SIZE PREFIXES AND SYMBOLS

SIZE	PREFIX	SYMBOL	SIZE	PREFIX	SYMBOL
1000 ¹	Kilo-	K	2 ¹⁰	Kibi-	Ki
1000 ²	Mega-	M	2 ²⁰	Mebi-	Mi
1000 ³	Giga-	G	2 ³⁰	Gibi-	Gi
1000 ⁴	Tera-	T	2 ⁴⁰	Tebi-	Ti
1000 ⁵	Peta-	P	2 ⁵⁰	Pebi-	Pi
1000 ⁶	Exa-	E	2 ⁶⁰	Exbi-	Ei
1000 ⁷	Zetta-	Z	2 ⁷⁰	Zebi-	Zi
1000 ⁸	Yotta-	Y	2 ⁸⁰	Yobi-	Yi
1000 ⁹	Ronna-	R	2 ⁹⁰	Robi-	Ri
1000 ¹⁰	Quecca-	Q	2 ¹⁰⁰	Quebi-	Qi
1000 ⁻¹	milli-	m	1000 ⁻³	femto-	f
1000 ⁻²	micro-	μ	1000 ⁻⁶	atto-	a
1000 ⁻³	nano-	n	1000 ⁻⁹	zepto-	z
1000 ⁻⁴	pico-	p	1000 ⁻¹²	yocto-	y
			1000 ⁻¹⁵	ronto-	r
			1000 ⁻¹⁸	quecto-	q

What is the *minimum* size of the activation record (i.e., stack frame) for each of the following functions (assuming no compiler optimizations)? Assume the RISC-V presented in class and the textbook (see above reference, however, you do NOT need to worry about more than word-alignment of the stack). Report your answers in bytes. Please enter your work and justification in the next question in case of partial credit.

```
int blahblah(int a, int b) {
    int c[ 256 ];
    for (int i=0; i<256; i++){
        c[ i ] = i;
    }
    return a*c[ blah(a) ]+b*c[ blah(b) ];
}

int blah(int a) {
    return a & 0xFF;
}
```

ANSWER: 1036 = 256*4 + return address (4) + a (4) + b (4) {need to save a/b for surviving blah calls}

ANSWER: 0 – no arrays, no internal function calls, no updating variables

Assuming that s0,s2, and t1 start out with 0x00000000, 0x0000000F, and 0xFFFFFFFF, what will they have after the execution of the following RISC-V assembly? Answer in full 32-bit hexadecimal of the form 0x01234567.

```
    j cond
loop:
    add t1, s0, s1
    lw  t1, 0(t1)
    add t0, t0, t1
    addi s0, s0, 4
cond:
    slt t1, s0, s2
    bne t1, zero, loop
```

s0: 0x00000010
s2: 0x0000000F
t1: 0x00000000
Instructions A: 27
Instructions B: 27

How many instructions are executed? You may simplify or leave it unsimplified without spaces (e.g., 100+42*381).
How many instructions are executed for initial values of 0xFFFFFFFF, 0x0000000F, and 0xFFFFFFFF? -1

How many *dynamic* instructions are executed by the below RISC-V code? Assume the following two's complement operation:

```
sub s0, s0, s0      # i=0
j Cond              # goto Cond
Loop: add t2, s2, s0 # t2 = &(b[i])
      add t3, s3, s0 # t3 = &(a[i])
      lb  t1, 0(t2)  # b[i]
      add t0, t1, s1  # t0 = b[i] + j
      sb  t0, 0(t3)   # a[i] = t0
      lb  t1, 1(t2)   # b[i+1]
      add t0, t1, s1  # t0 = b[i+1] + j
      sb  t0, 1(t3)   # a[i+1] = t0
      lb  t1, 2(t2)   # b[i+2]
      add t0, t1, s1  # t0 = b[i+2] + j
      sb  t0, 2(t3)   # a[i+2] = t0
      lb  t1, 3(t2)   # b[i+3]
      add t0, t1, s1  # t0 = b[i+3] + j
      sb  t0, 3(t3)   # a[i+3] = t0
      addi s0, s0, 4  # i+=4
Cond: slti t0, s0, 32 # (i<32)
      bne t0, zero, Loop # goto Loop if i<32
```

4 + 17*8 = 140

A + B
Can overflow occur when A is zero? No
Can overflow occur when B is zero? No
Now assume the following two's complement operation:
A - B
Can overflow occur when A is zero? ~~No~~ Yes
Can overflow occur when B is zero? No
Does RISC-V require hardware to check overflow? No
How many *dynamic* instructions are executed by the below RISC-V code? Assume a0 is 4.

```
fact: slti t0, a0, 2      # a0 < 2
      beq t0, zero, skip  # goto skip
      ori a0, zero, 1     # Return 1
      jr  ra              # Return
skip: addi sp, sp, -8      # sp down 32
      sw  ra, 0(sp)       # Save ra
      sw  a0, 4(sp)       # Save n
      addi a0, a0, -1     # n - 1
      jal fact            # Call fact
link: lw  a1, 4(sp)       # Restore n
      mul a0, a1, a0      # n*fact(n-1)
      lw  ra, 0(sp)       # Load $ra
      addi sp, sp, 8      # Pop stack
      jr  ra              # Return
```

Pass1: 4 – 2in – stck: 0
Pass2: 3 – 9in – stck: 8
Pass3: 2 – 16in – stck: 16
Pass4: 1 – 23in – stck: 24
Link1: stck: 24 – 25+5in
Link2: stck: 16 – 30+5in
Link3: stck: 8 – 35+5in
Stack: -24: Link, -20: 2, -16: Link, -12: 3, -8: Caller, -4: 4
Mul1: a0 = 2*1
Mul2: a0 = 3*2 = 6
Mul3: a0 = 4*6 = 24

What value (in 0x??????? format) does t0 have after the following instruction sequence?

```
lui t0, 0x321D1
addi t0, t0, -15
slli t0, t0, 24
srai t0, t0, 24
```

0x321D1000
0x321D1000 + 0xFFFFFFFF
0x321F0FF1 << 24 = 0xF1000000
0xF1000000 >> 24 = 0xFFFFFFFF

0xFFFFFFFF

What are the final values (in hexadecimal beginning with 0x) of registers t0 and t1? What are the final values of memory at addresses 0x10010000 and 0x10010001 (initial values 0x00000000)?

```
lui t1, 0x10010
lb t0, 0(t1)
ori t0, t0, 0x7F0
sb t0, 0(t1)
```

t1 = 0x10010000
t0 = M[0x10010000]
t0 = 0x000007F0
M[0x10010000] = 0x000000F0
M[0x10010001] = 0x00000000

How many dynamic instructions are executed by the below RISC-V code?

More registers have the following impacts:

```
sub    s0, s0, s0    # i=0
j      Cond          # goto Cond
Loop:  add    t2, s2, s0    # t2 = &(b[i])
      add    t3, s3, s0    # t3 = &(a[i])
      lb     t1, 0(t2)     # b[i]
      add    t0, t1, s1    # t0 = b[i] + j
      sb     t0, 0(t3)     # a[i] = t0
      lb     t1, 1(t2)     # b[i+1]
      add    t0, t1, s1    # t0 = b[i+1] + j
      sb     t0, 1(t3)     # a[i+1] = t0
      lb     t1, 2(t2)     # b[i+2]
      add    t0, t1, s1    # t0 = b[i+2] + j
      sb     t0, 2(t3)     # a[i+2] = t0
      lb     t1, 3(t2)     # b[i+3]
      add    t0, t1, s1    # t0 = b[i+3] + j
      sb     t0, 3(t3)     # a[i+3] = t0
      addi   s0, s0, 4      # i+=4
Cond:  slti   t0, s0, 32    # (i<32)
      bne    t0, zero, Loop    # goto Loop if i<32
```

$$21 + 17 \times 8 = 138$$

☒ Fewer spills to and fills from memory

☐ More spills to and fills from memory

☐ Longer access latency

☒ Shorter access latency

☐ Require more bits in instructions

☐ No impact on the ISA

☒ Easier to compile for

Consider the following loop:

```
0x1000: Loop: addi t3, s3, 1
0x1004: lw     s2, 0(t3)
0x1008: bne    t2, s2, Loop
0x100c: ...
```

What should the branch's immediate value be? (total immediate in appropriately sized binary)

111111111000

Can you update the ABI to keep the lack of a hardware-supported zero register from being a problem even for legacy programs? Why or why not?

It depends on if the legacy code includes instructions that assume and need x0 to be 0. If these were included in the code, no, it can't be updated without including new instructions. If they're not, it could potentially.